DATA ANALYTICS • SQL

SQL Project — Retail Store Analytics

A portfolio-ready SQL project focused on relational data, business questions, multi-table analysis and practical query writing.

PROJECT TYPE	DOMAIN	FOCUS
SQL Mini Project	Retail / E-commerce	Data Analysis

PROJECT INTRODUCTION & PORTFOLIO SECTION

01 | PROJECT AIM

From raw retail data to business-ready insight

The aim of this project is to use SQL to transform a structured retail dataset into clear, business-oriented answers. The supplied project asks the learner to create the tables, load the data and construct SQL queries for practical business questions.

STEP 01

Build

Create the relational structure using the supplied DDL scripts.

STEP 02

Load

Populate the tables using the supplied DML scripts.

STEP 03

Query

Write SQL queries that answer the business questions.

STEP 04

Interpret

Connect query outputs with real retail use cases.

Why this project adds portfolio value

1

Relational thinking

Understanding entities, primary keys and foreign keys.

2

Query building

Progression from filtering to aggregations, joins and subqueries.

3

Business orientation

Questions are connected to practical retail scenarios.

4

Problem-solving

Shows a structured schema → query → result workflow.

Everything the project puts into practice

The project is centered on **SQL and relational database concepts**. The supplied DDL includes primary keys, AUTO_INCREMENT, foreign keys, DECIMAL values, CHECK constraints and date/time fields.

SQL / RDBMS

Foundations

Relational database design

DDL + DML · Data Query Language

Core Query Skills

Retrieval & formatting

SELECT · WHERE · BETWEEN · IN · LIKE

ORDER BY · LIMIT · NULLs

Aliases · calculations · dates

Analytics

Aggregation

COUNT · SUM · AVG · MIN · MAX

GROUP BY · HAVING

Advanced SQL

Multi-table & nested logic

INNER / LEFT / RIGHT JOIN

Multiple joins · subqueries

EXISTS / NOT EXISTS · UNION



Six entities model a complete retail operation

The retail model covers customers, products, orders, order-line items, payments and product reviews. The supplied table structures define the fields, constraints and real-world meaning of these entities.

ENTITY	PRIMARY KEY	KEY FIELDS
Customers	PK customer_id	name, email, phone, created_at
Products	PK product_id	name, category, price, stock_quantity, added_on
Orders	PK order_id	customer_id, order_date, status, total_amount
Order Items	PK order_item_id	order_id, product_id, quantity, item_price
Payments	PK payment_id	order_id, payment_date, amount_paid, method
Product Reviews	PK review_id	product_id, customer_id, rating, review_text, review_date

Relationship map

1 → N **CUSTOMERS → ORDERS** A **customer** can place multiple **orders**.

1 → N **ORDERS → ORDER ITEMS** An **order** can contain multiple **items**.

1 → N **PRODUCTS → ORDER ITEMS** A **product** can appear in many **orders**.

1 → N **ORDERS → PAYMENTS** The source specifies one or more **payments** per order.

1 → N **PRODUCTS → REVIEWS** A **product** can have many customer **reviews**.

1 → N **CUSTOMERS → REVIEWS** A **customer** can write multiple **reviews**.

Schema note: the later schema, DDL and ER sections also include Product Reviews.

04 | BUSINESS QUESTION ROADMAP

42 questions, six levels of analytical depth

The project moves from basic retrieval to increasingly analytical SQL: filtering, calculations, aggregations, joins, subqueries and set operations.

42	6	6
BUSINESS QUESTIONS	SQL LEVELS	CORE ENTITIES

01	**Basics** Retrieval, filtering, categories and sorting	10 questions
02	**Filtering & Formatting** NULLs, aliases, calculations, dates and stock	6 questions
03	**Aggregations** Orders, revenue, AOV, customers, categories and payments	10 questions
04	**Multi-Table Queries** INNER / LEFT / RIGHT JOIN and 3-table reporting	7 questions
05	**Subqueries** Average comparisons, inactive customers and highest orders	7 questions
06	**Set Operations** Customer engagement across orders and reviews	2 questions

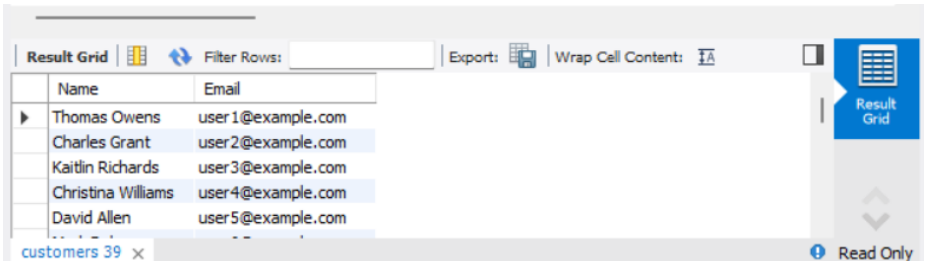
RECOMMENDED READING FLOW

SCHEMA → QUERY → RESULT → BUSINESS MEANING

LEVEL 1 BASICS

1. Retrieve customer names and emails for email marketing

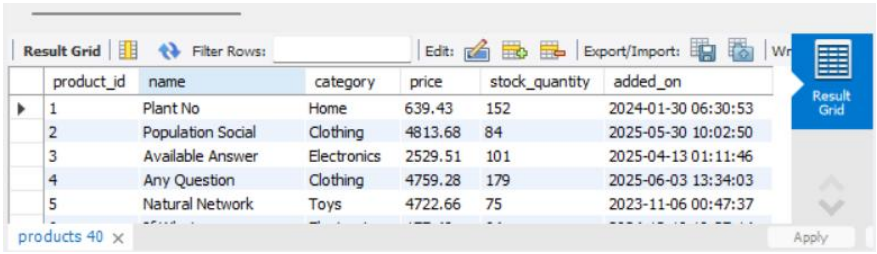
select Name , Email from customers;



Name	Email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com

2. View complete product catalog with all available details

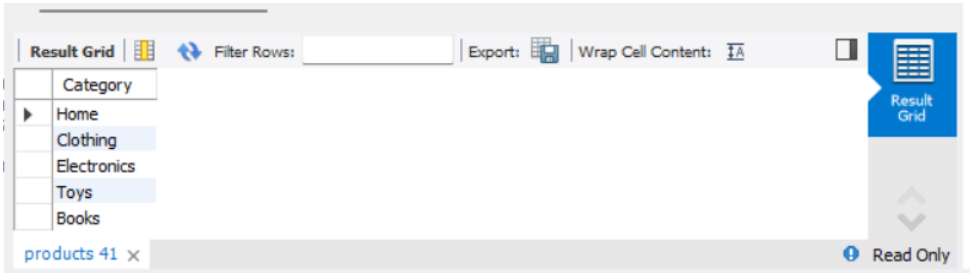
select * from products;



product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37

3. List all unique product categories

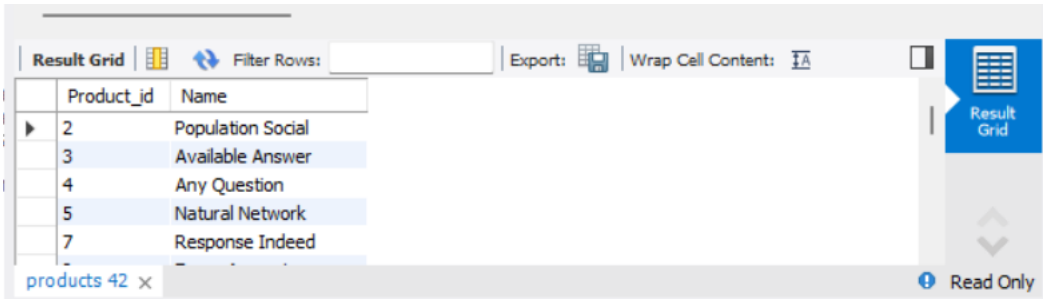
select distinct category as Category from products;



Category
Home
Clothing
Electronics
Toys
Books

4. Show all products priced above ₹1,000

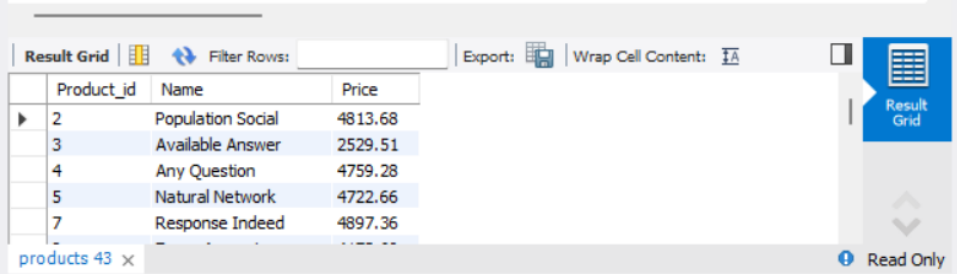
select Product_id,Name from products where price>1000;



Product_id	Name
2	Population Social
3	Available Answer
4	Any Question
5	Natural Network
7	Response Indeed

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

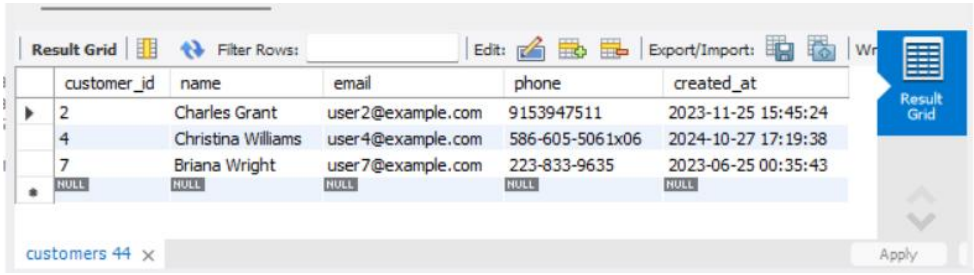
select Product_id,Name,Price from products where price between 2000 and 5000;



Product_id	Name	Price
2	Population Social	4813.68
3	Available Answer	2529.51
4	Any Question	4759.28
5	Natural Network	4722.66
7	Response Indeed	4897.36

6. Fetch data for specific customer IDs

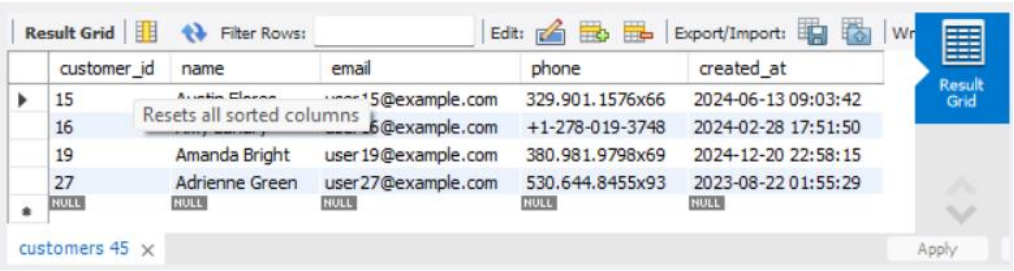
select * from customers where customer_id in (2,4,7);



customer_id	name	email	phone	created_at
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43

7. Identify customers whose names start with the letter ‘A’

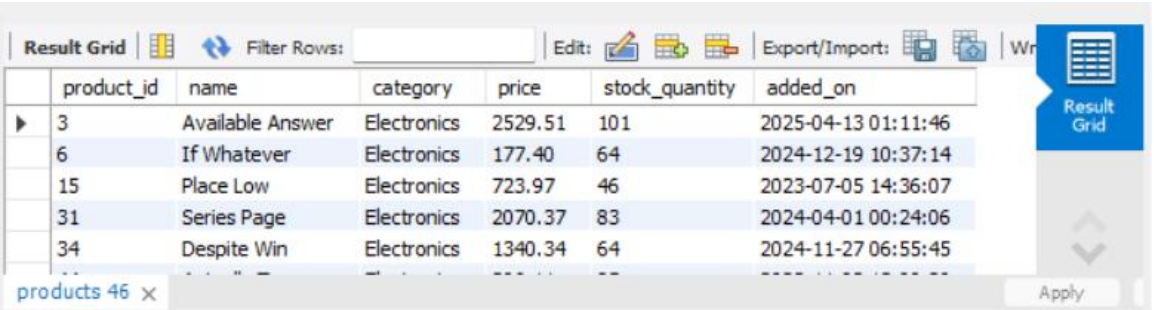
select * from customers where name like "A%";



customer_id	name	email	phone	created_at
15	[illegible]	[illegible]5@example.com	329.901.1576x66	2024-06-13 09:03:42
16	[illegible]	[illegible]6@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29

8. List electronics products priced under ₹3,000

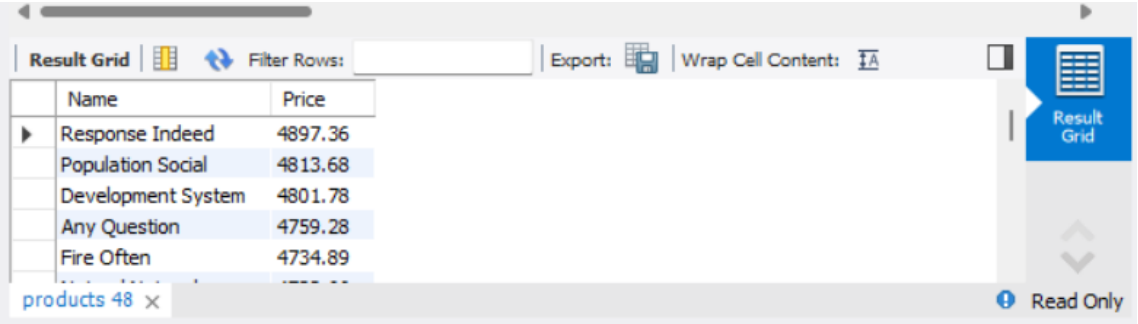
select * from products where category="Electronics" and price<=3000;



product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45

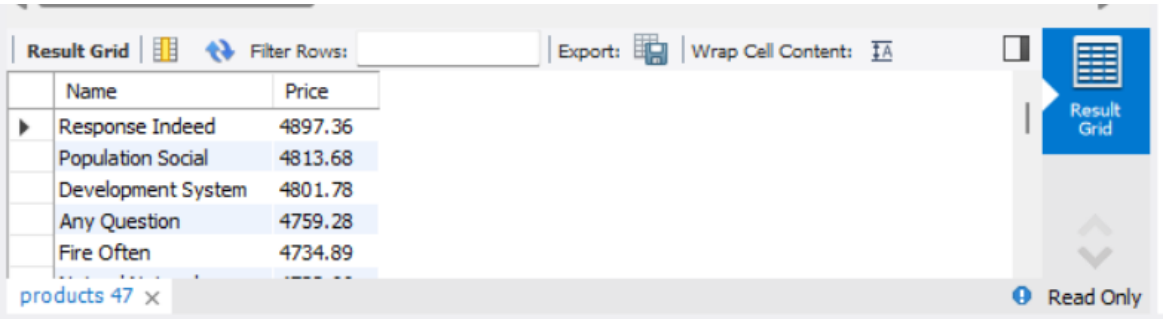
9. Display product names and prices in descending order of price

select Name , Price from products order by price desc;



10. Display product names & prices, sorted by price,then by name

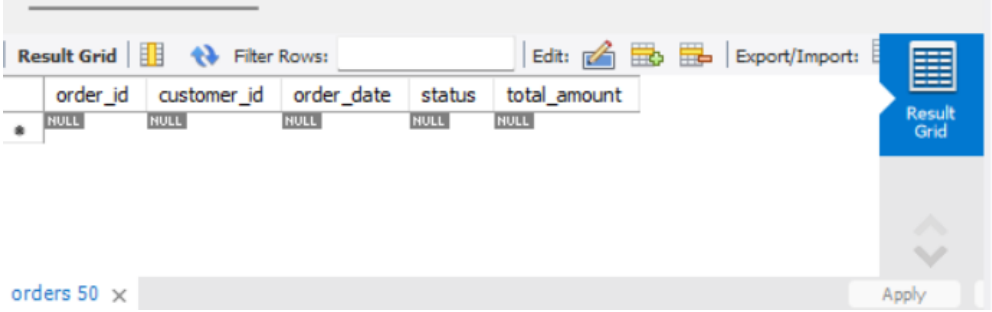
select Name , Price from products order by price desc,name asc;



LEVEL 2 FILTERRING AND FORMATTING

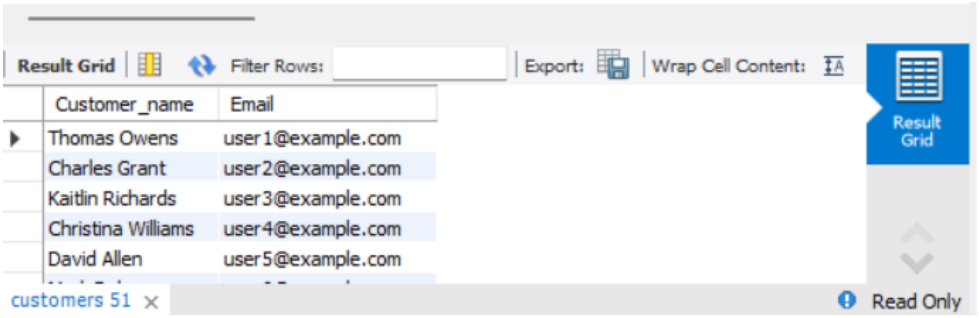
1. Retrieve orders where customer information is missing

select * from orders where customer_id is null;



2. Display customer names and emails using column aliases for frontend readability

Select name as Customer_name , email as Email from customers;



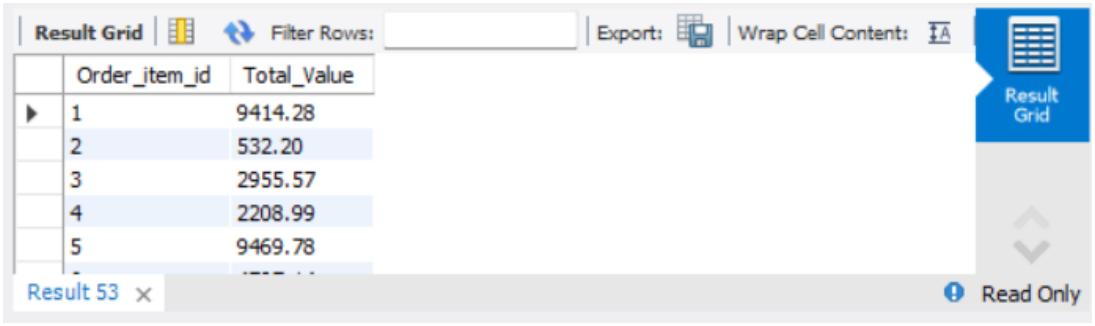
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_name	Email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com

customers 51 | Read Only

3. Calculate total value per item ordered by multiplying quantity and item price

select Order_item_id,quantity*item_price as Total_Value from order_items;



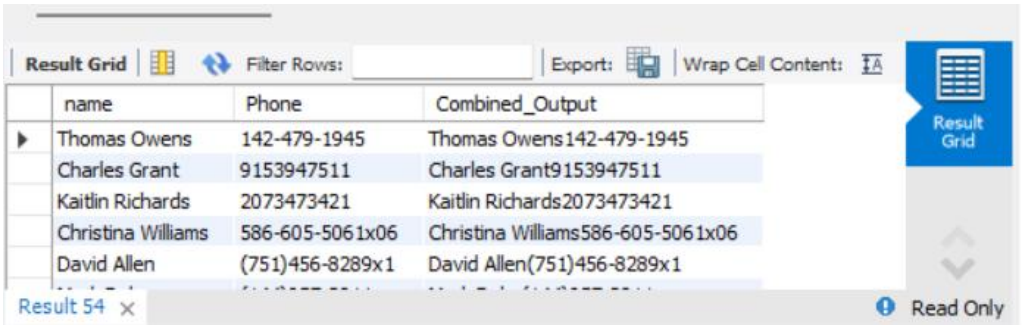
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_item_id	Total_Value
1	9414.28
2	532.20
3	2955.57
4	2208.99
5	9469.78

Result 53 | Read Only

4. Combine customer name and phone number in a single column

select name,Phone,concat(Name,"",Phone) as Combined_Output from customers;



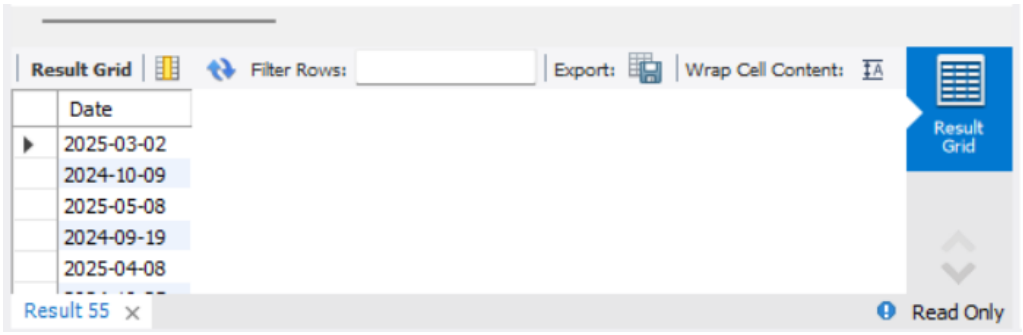
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	Phone	Combined_Output
Thomas Owens	142-479-1945	Thomas Owens142-479-1945
Charles Grant	9153947511	Charles Grant9153947511
Kaitlin Richards	2073473421	Kaitlin Richards2073473421
Christina Williams	586-605-5061x06	Christina Williams586-605-5061x06
David Allen	(751)456-8289x1	David Allen(751)456-8289x1

Result 54 | Read Only

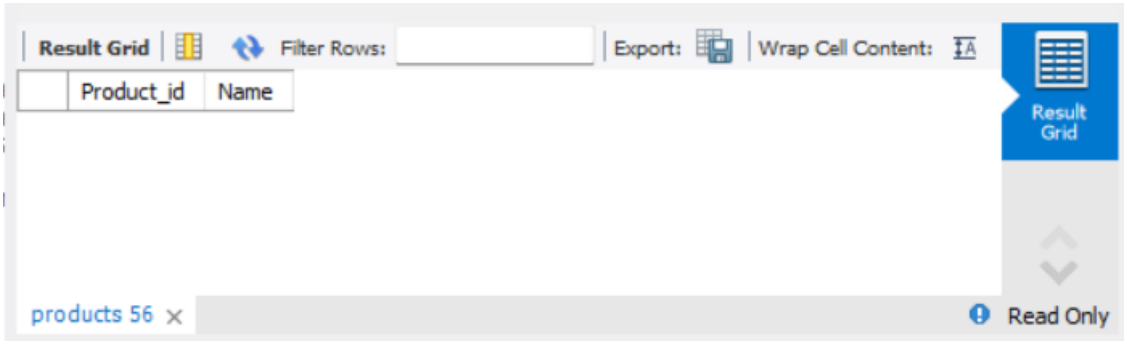
5. Extract only the date part from order timestamps for date-wise reporting

select date(order_date) as Date from orders;



6. List products that do not have any stock left

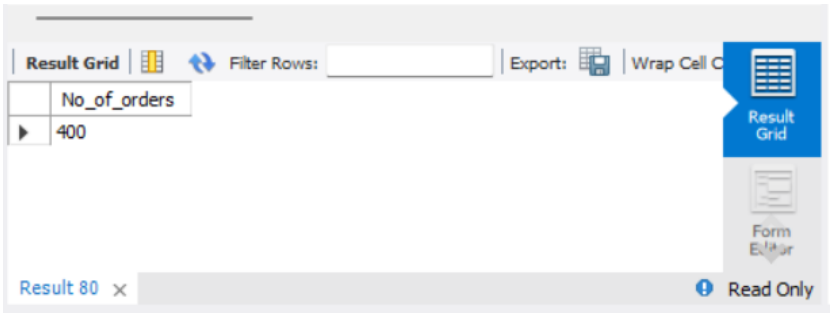
select Product_id,Name from products where stock_quantity =0;



LEVEL 3 AGGREGATIONS

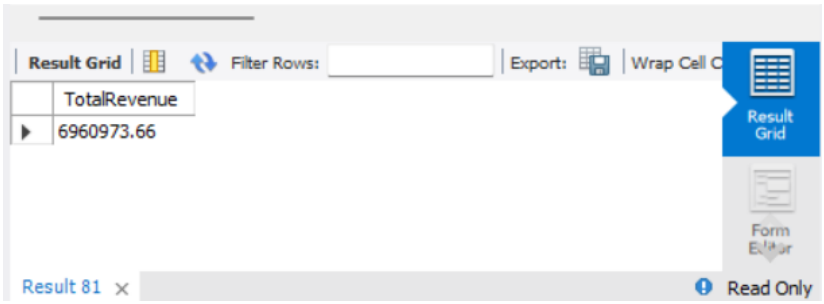
1. Count the total number of orders placed

select count(order_id) as No_of_orders from orders;



2. Calculate the total revenue collected from all orders

select sum(total_amount) as TotalRevenue from orders;



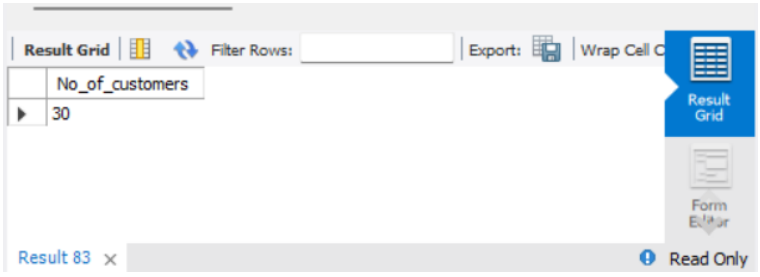
3. Calculate the average order value

select avg(total_amount) as AvgOrderValue from orders;



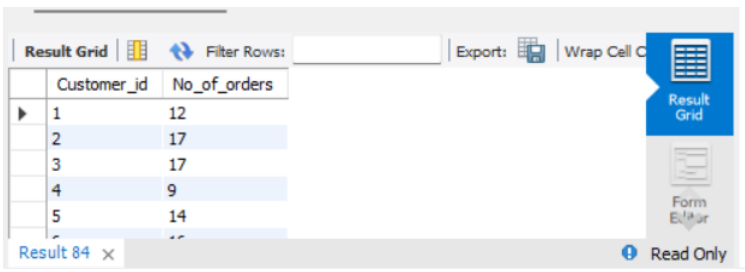
4. Count the number of customers who have placed at least one order

select count(distinct customer_id) as No_of_customers from orders ;



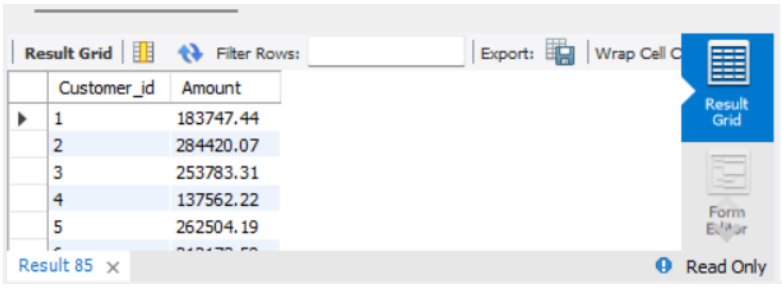
5. Find the number of orders placed by each customer

select Customer_id, count(order_id) as No_of_orders from orders group by customer_id;



6. Find total sales amount made by each customer

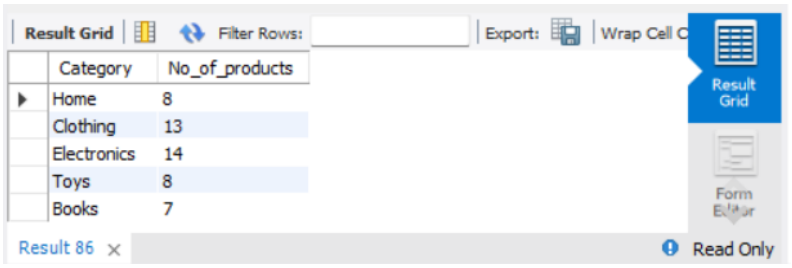
select Customer_id, sum(total_amount) as Amount from orders group by customer_id;



Customer_id	Amount
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19

7. List the number of products sold per category

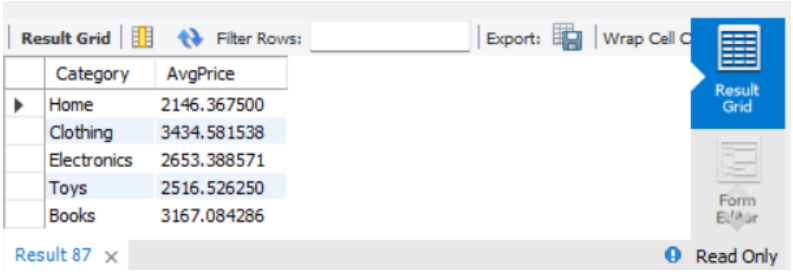
select Category, count(product_id) as No_of_products from products group by category;



Category	No_of_products
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

8. Find the average item price per category

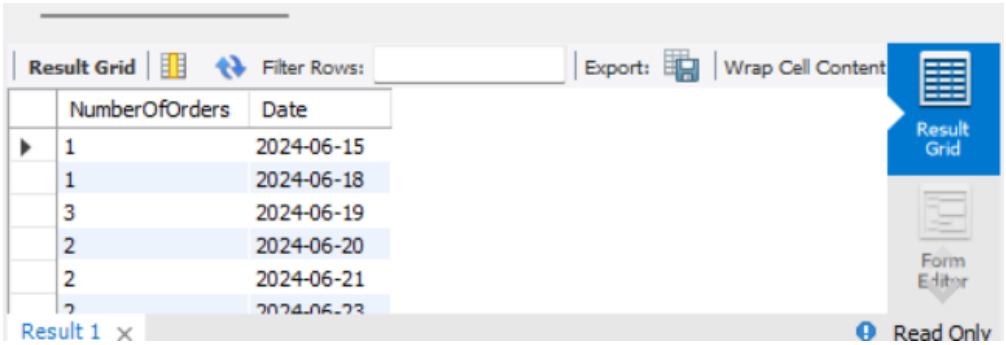
select Category,avg(price) as AvgPrice from products group by Category;



Category	AvgPrice
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

9. Show number of orders placed per day

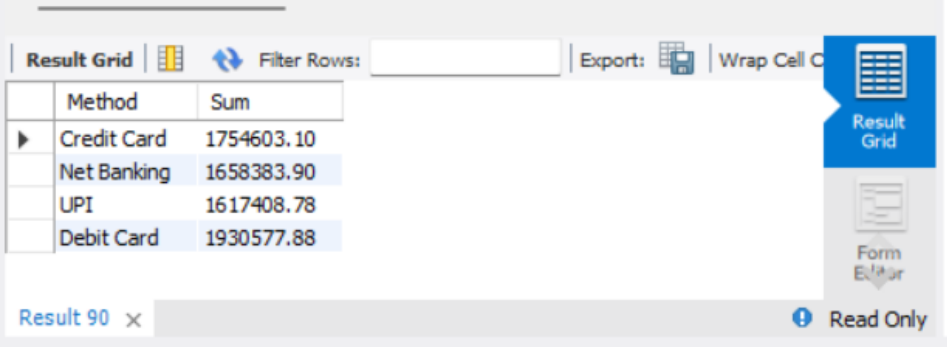
select count(distinct order_id) as NumberOfOrders, date(Order_Date) as Date from orders group by date(order_date);



NumberOfOrders	Date
1	2024-06-15
1	2024-06-18
3	2024-06-19
2	2024-06-20
2	2024-06-21

10. List total payments received per payment method

select Method ,sum(amount_paid) as Sum from payments group by Method;



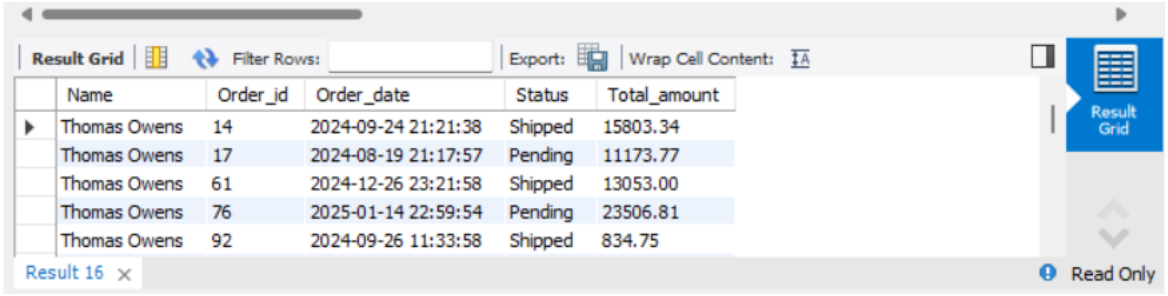
Method	Sum
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

Result 90 | Read Only

LEVEL 4 MULTI TABLE QUERIES (JOINS)

1. Retrieve order details along with the customer name (INNER join)

select c.Name ,o.Order_id,o.Order_date,o.Status,o.Total_amount from customers as c inner join orders as o on c.customer_id=o.customer_id;

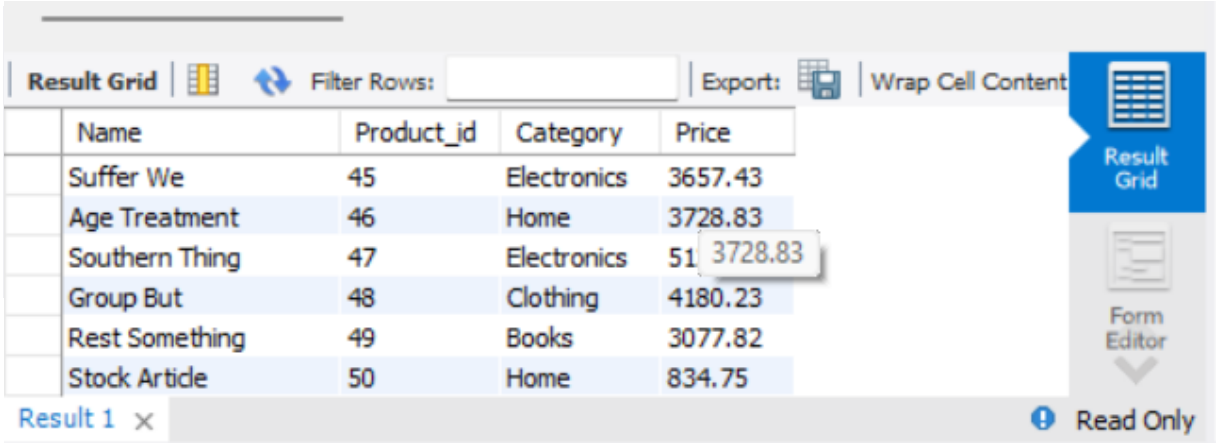


Name	Order_id	Order_date	Status	Total_amount
Thomas Owens	14	2024-09-24 21:21:38	Shipped	15803.34
Thomas Owens	17	2024-08-19 21:17:57	Pending	11173.77
Thomas Owens	61	2024-12-26 23:21:58	Shipped	13053.00
Thomas Owens	76	2025-01-14 22:59:54	Pending	23506.81
Thomas Owens	92	2024-09-26 11:33:58	Shipped	834.75

Result 16 | Read Only

2. Get list of products that have been sold (INNER JOIN with order_items)

select distinct p.Name,p.Product_id ,p.Category,p.Price from products as p inner join order_items as oi on p.product_id=oi.product_id;



Name	Product_id	Category	Price
Suffer We	45	Electronics	3657.43
Age Treatment	46	Home	3728.83
Southern Thing	47	Electronics	51
Group But	48	Clothing	4180.23
Rest Something	49	Books	3077.82
Stock Article	50	Home	834.75

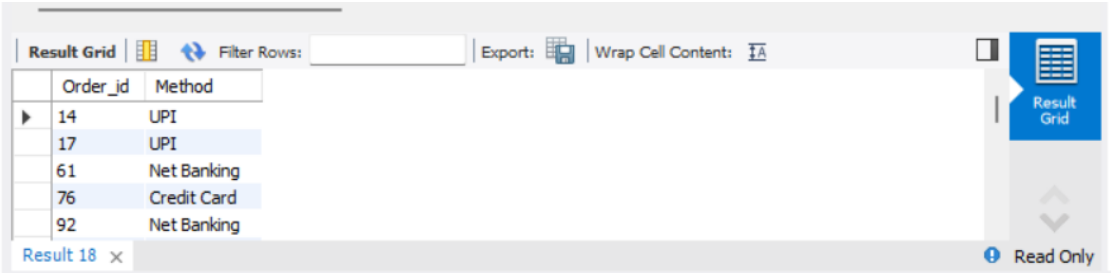
3728.83

Result 1 | Read Only

3. List all orders with their payment method (INNER JOIN)

select o.Order_id,p.Method from orders as o inner join payments as p

on o.order_id=p.order_id;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

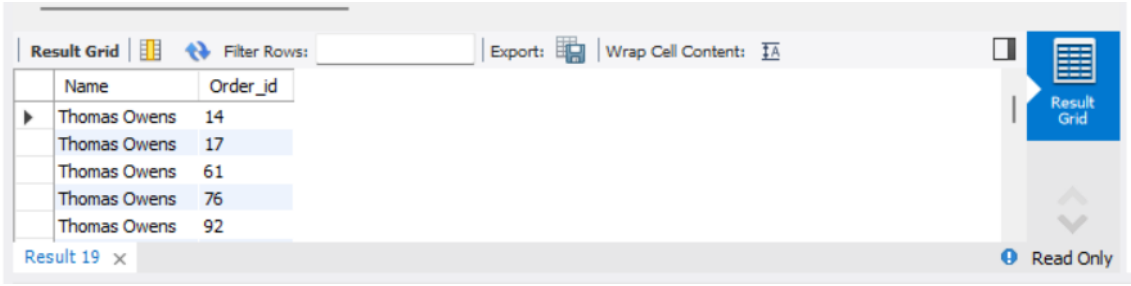
Order_id	Method
14	UPI
17	UPI
61	Net Banking
76	Credit Card
92	Net Banking

Result 18 | Read Only

4. Get list of customers and their orders (LEFT JOIN)

select c.Name,o.Order_id from customers as c left join orders as o

on c.Customer_id=o.customer_id ;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

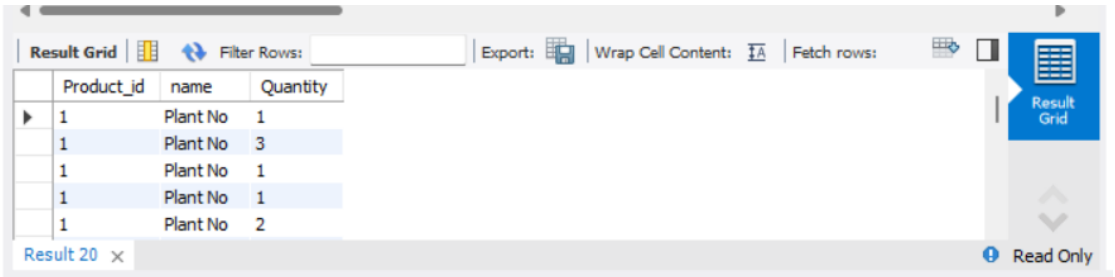
Name	Order_id
Thomas Owens	14
Thomas Owens	17
Thomas Owens	61
Thomas Owens	76
Thomas Owens	92

Result 19 | Read Only

5. List all products along with order item quantity (LEFT JOIN)

select p.Product_id,p.name ,o.Quantity from products as p left join order_items as o

on p.product_id=o.product_id;



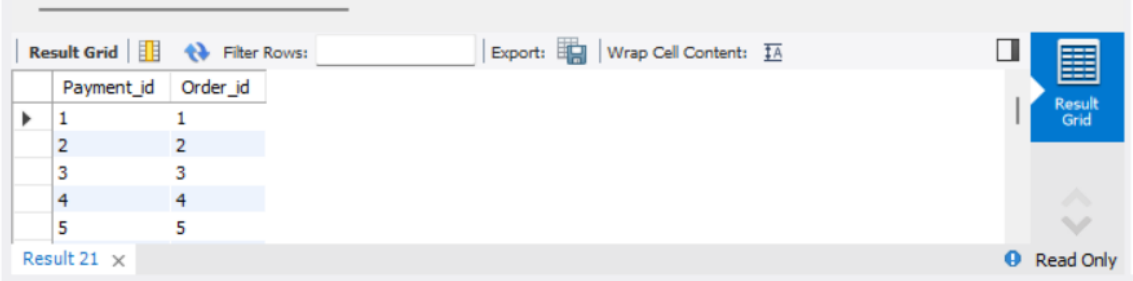
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Product_id	name	Quantity
1	Plant No	1
1	Plant No	3
1	Plant No	1
1	Plant No	1
1	Plant No	2

Result 20 | Read Only

6. List all payments including those with no matching orders (RIGHT JOIN)

select p.Payment_id,o.Order_id from orders as o right join payments as p on o.order_id =p.order_id;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

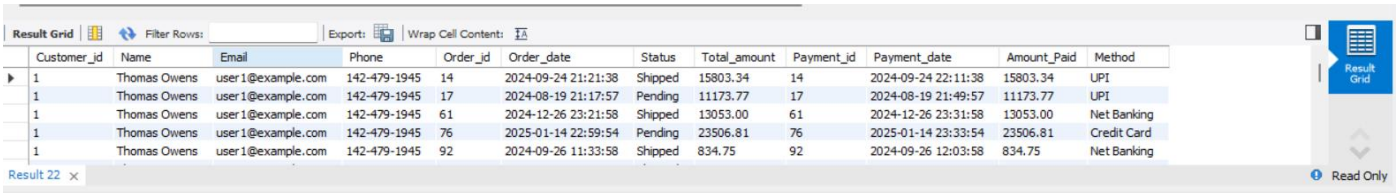
Payment_id	Order_id
1	1
2	2
3	3
4	4
5	5

Result 21 | Read Only

7. Combine data from three tables: customer, order, and payment

select
c.Customer_id,c.Name,c.Email,c.Phone,o.Order_id,o.Order_date,o.Status,o.Total_amount,p.Payment_id,

p.Payment_date,p.Amount_Paid,p.Method from customers as c inner join orders as o on
c.customer_id=o.customer_id inner join payments as p on o.order_id=p.order_id;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

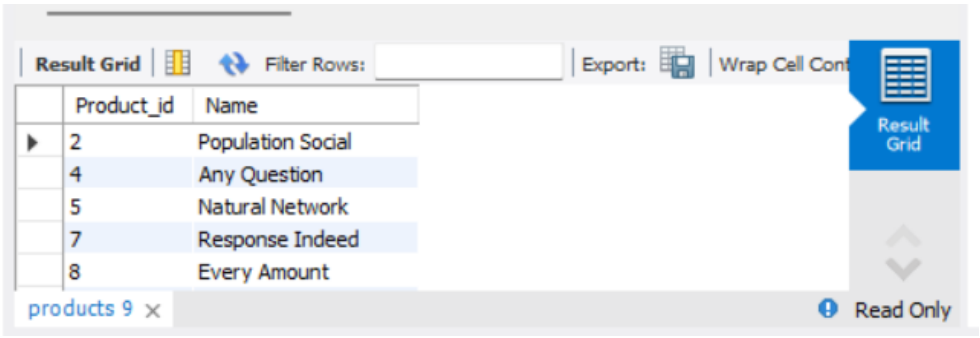
Customer_id	Name	Email	Phone	Order_id	Order_date	Status	Total_amount	Payment_id	Payment_date	Amount_Paid	Method
1	Thomas Owens	user1@example.com	142-479-1945	14	2024-09-24 21:21:38	Shipped	15803.34	14	2024-09-24 22:11:38	15803.34	UPI
1	Thomas Owens	user1@example.com	142-479-1945	17	2024-08-19 21:17:57	Pending	11173.77	17	2024-08-19 21:49:57	11173.77	UPI
1	Thomas Owens	user1@example.com	142-479-1945	61	2024-12-26 23:21:58	Shipped	13053.00	61	2024-12-26 23:31:58	13053.00	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	76	2025-01-14 22:59:54	Pending	23506.81	76	2025-01-14 23:33:54	23506.81	Credit Card
1	Thomas Owens	user1@example.com	142-479-1945	92	2024-09-26 11:33:58	Shipped	834.75	92	2024-09-26 12:03:58	834.75	Net Banking

Result 22 | Read Only

LEVEL 5 SUBQUERIES

1. List all products priced above the average product price

select Product_id,Name from products where price>(select avg(price) from products);



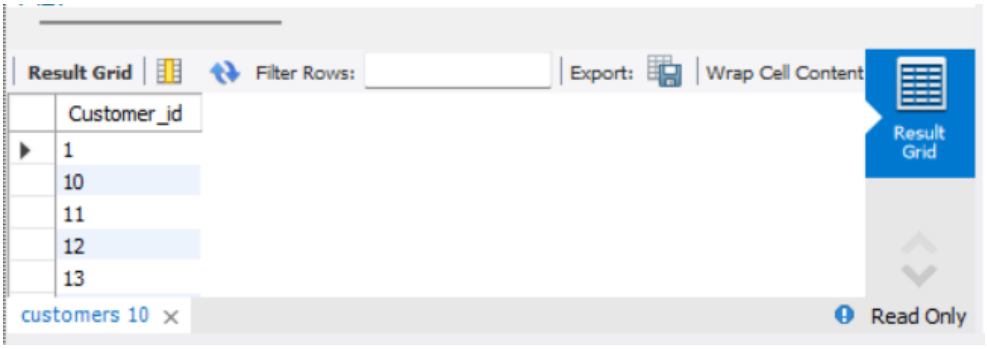
Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Product_id	Name
2	Population Social
4	Any Question
5	Natural Network
7	Response Indeed
8	Every Amount

products 9 | Read Only

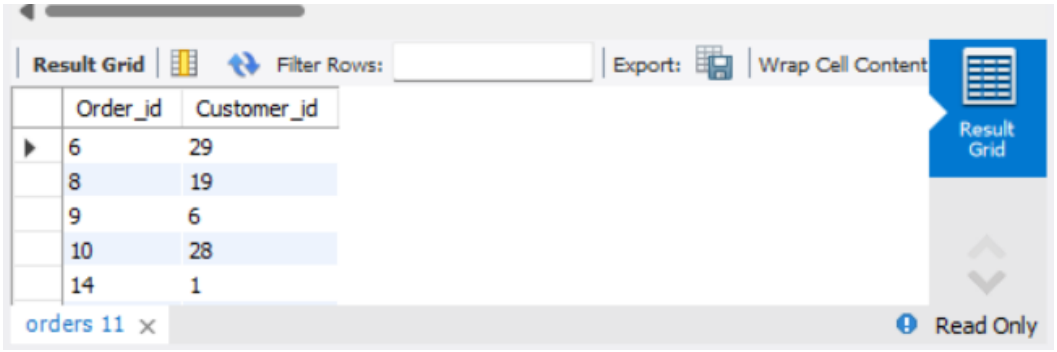
2. Find customers who have placed at least one order

select Customer_id from customers where customer_id in (select Customer_id from Orders);



3. Show orders whose total amount is above the average for that customer

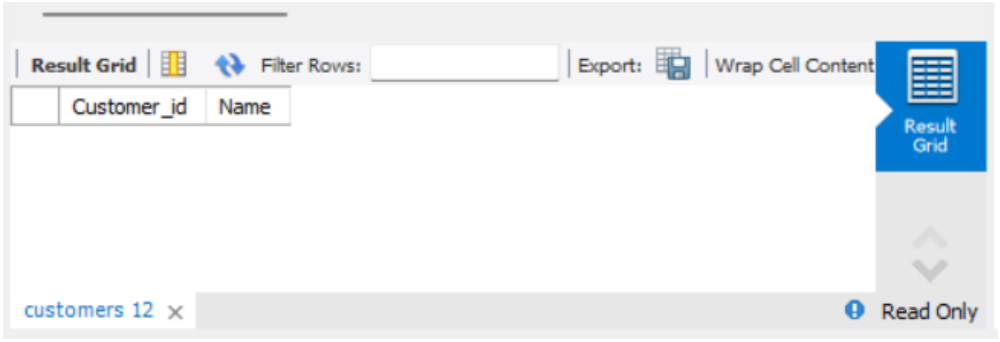
select o.Order_id,o.Customer_id from orders o where o.Total_amount >(select avg(o2.total_amount) from orders o2 where o2.Customer_id = o.Customer_id);



4. Display customers who haven't placed any orders

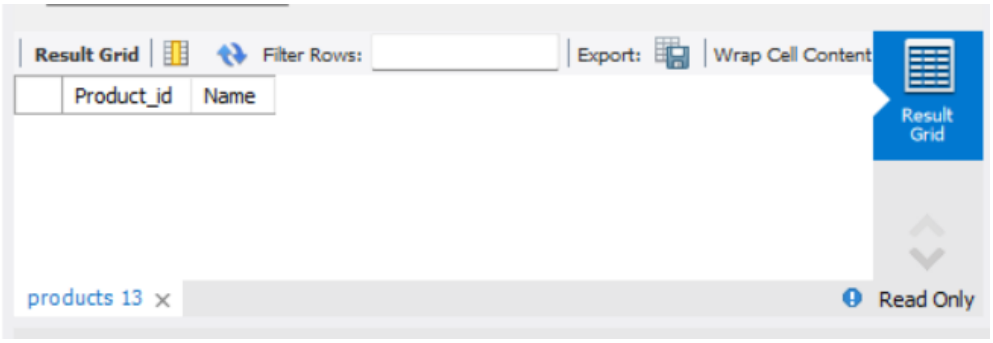
select c.Customer_id, c.Name from customers c where not exists (

select * from orders o where o.Customer_id = c.Customer_id);



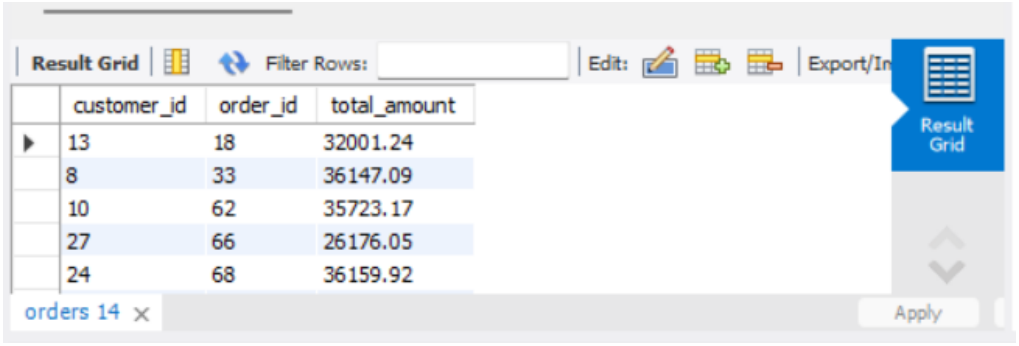
5. Show products that were never ordered

select p.Product_id,p.Name from products p where not exists(select * from order_items o where p.Product_id=o.product_id);



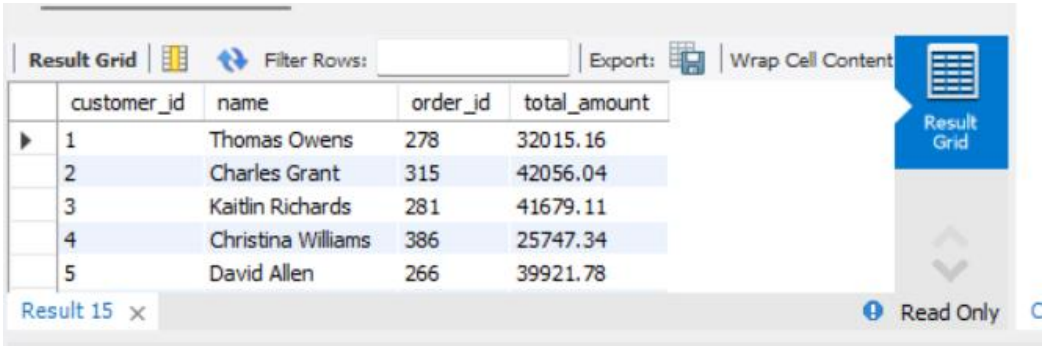
6. Show highest value order per customer

select o.customer_id, o.order_id, o.total_amount from orders o where o.total_amount = (select MAX(o2.total_amount) from orders o2 where o2.customer_id = o.customer_id);



7. Highest Order Per Customer (Including Names)

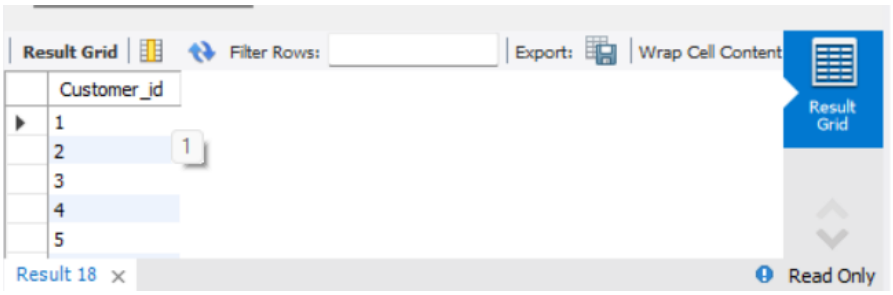
select o.customer_id,c.name, o.order_id, o.total_amount from orders o inner join customers c on c.customer_id=o.customer_id where o.total_amount = (select MAX(o2.total_amount) from orders o2 where o2.customer_id = o.customer_id);



LEVEL 6 SET OPERATIONS

1. List all customers who have either placed an order or written a product review

```
select Customer_id from orders union
select Customer_id from product_reviews;
```



2. List all customers who have placed an order as well as reviewed a product

```
select distinct customer_id
from orders
where customer_id in (
   select customer_id
   from product_reviews
);
```

